Guard App — Screens

Every screen in the guard app with full TypeScript/TSX component code.

---

1. Welcome Screen

```
// app/(auth)/welcome.tsx
import { View, Text, Image, Pressable } from 'react-native';
import { useRouter } from 'expo-router';
import { StatusBar } from 'expo-status-bar';

export default function WelcomeScreen() {
  const router = useRouter();

  return (
    <View className="flex-1 bg-green-950">
      <StatusBar style="light" />
      <View className="flex-1 items-center justify-center px-8">
        <Image
          source={require('@/assets/images/guard-logo.png')}
          className="w-32 h-32 mb-8"
          resizeMode="contain"
        />
        <Text className="text-white text-4xl font-bold text-center
mb-2">
          b-secure Guard
        </Text>
        <Text className="text-green-300 text-lg text-center
mb-12">
          Professional security services platform
        </Text>
      </View>

      <View className="px-8 pb-12 gap-4">
        <Pressable
          onPress={() => router.push('/(auth)/login')}
          className="bg-green-500 rounded-2xl py-4 items-center
active:opacity-80"
        >
          <Text className="text-white text-lg font-semibold">Get
Started</Text>
        </Pressable>
        <Text className="text-green-400 text-center text-sm">
          For registered security professionals only
```

```
        </Text>
      </View>
    </View>
  );
}
```

---

2. Login Screen

```
// app/(auth)/login.tsx
import { useState } from 'react';
import { View, Text, TextInput, Pressable, KeyboardAvoidingView,
Platform } from 'react-native';
import { useRouter } from 'expo-router';
import { useMutation } from '@tanstack/react-query';
import { authService } from '@/services/api/authService';

export default function LoginScreen() {
  const router = useRouter();
  const [phone, setPhone] = useState('');

  const requestOtp = useMutation({
    mutationFn: (phone: string) =>
authService.requestGuardOtp(phone),
    onSuccess: () => {
      router.push({ pathname: '/(auth)/otp-verify', params:
{ phone } });
    },
  });

  const isValid = phone.replace(/\D/g, '').length === 10;

  return (
    <KeyboardAvoidingView
      behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
      className="flex-1 bg-white"
    >
      <View className="flex-1 px-6 pt-20">
        <Text className="text-3xl font-bold text-gray-900 mb-2">
          Welcome back
        </Text>
        <Text className="text-gray-500 mb-10">
          Enter your registered mobile number to continue
        </Text>

        <View className="flex-row items-center border border-
gray-300 rounded-xl px-4 py-3 mb-4">
          <Text className="text-gray-600 text-lg mr-2">+91</Text>
          <View className="w-px h-6 bg-gray-300 mr-2" />
          <TextInput
            value={phone}
            onChangeText={setPhone}
```

```
            placeholder="10-digit mobile number"
            keyboardType="phone-pad"
            maxLength={10}
            className="flex-1 text-lg text-gray-900"
            autoFocus
          />
        </View>

        {requestOtp.isError && (
          <Text className="text-red-500 text-sm mb-4">
            {(requestOtp.error as any)?.message || 'Failed to send
OTP. Try again.'}
          </Text>
        )}

        <Pressable
          onPress={() => requestOtp.mutate(phone)}
          disabled={!isValid || requestOtp.isPending}
          className={`rounded-xl py-4 items-center ${
            isValid && !requestOtp.isPending
              ? 'bg-green-600 active:opacity-80'
              : 'bg-gray-200'
          }`}
        >
          <Text
            className={`text-lg font-semibold ${
              isValid && !requestOtp.isPending ? 'text-white' :
'text-gray-400'
            }`}
          >
            {requestOtp.isPending ? 'Sending OTP...' : 'Send OTP'}
          </Text>
        </Pressable>
      </View>
    </KeyboardAvoidingView>
  );
}
```

---

3. OTP Verify Screen

```
// app/(auth)/otp-verify.tsx
import { useState, useRef, useEffect } from 'react';
import { View, Text, TextInput, Pressable } from 'react-native';
import { useLocalSearchParams, useRouter } from 'expo-router';
import { useMutation } from '@tanstack/react-query';
import { authService } from '@/services/api/authService';
import { useGuardStore } from '@/stores/guardStore';
import { storeToken } from '@/lib/storage';

export default function OtpVerifyScreen() {
  const { phone } = useLocalSearchParams<{ phone: string }>();
```

```
  const router = useRouter();
  const { setGuard } = useGuardStore();
  const [otp, setOtp] = useState(['', '', '', '', '', '']);
  const [resendTimer, setResendTimer] = useState(30);
  const inputRefs = useRef<TextInput[]>([]);

  useEffect(() => {
    if (resendTimer > 0) {
      const timer = setTimeout(() => setResendTimer((t) => t - 1),
1000);
      return () => clearTimeout(timer);
    }
  }, [resendTimer]);

  const verifyOtp = useMutation({
    mutationFn: (code: string) =>
      authService.verifyGuardOtp(phone, code),
    onSuccess: async ({ token, guard }) => {
      await storeToken(token);
      setGuard(guard);

      if (guard.verification_status === 'unverified') {
        router.replace('/onboarding/personal-info');
      } else if (
        guard.verification_status === 'pending' ||
        guard.verification_status === 'under_review'
      ) {
        router.replace('/pending-approval');
      } else {
        router.replace('/(tabs)/dashboard');
      }
    },
  });

  const handleOtpChange = (value: string, index: number) => {
    const newOtp = [...otp];
    newOtp[index] = value;
    setOtp(newOtp);

    if (value && index < 5) {
      inputRefs.current[index + 1]?.focus();
    }

    if (newOtp.every((d) => d !== '') && newOtp.join('').length
=== 6) {
      verifyOtp.mutate(newOtp.join(''));
    }
  };

  return (
    <View className="flex-1 bg-white px-6 pt-20">
      <Text className="text-3xl font-bold text-gray-900 mb-2">
        Enter OTP
      </Text>
```

```
      <Text className="text-gray-500 mb-10">
        Sent to +91 {phone}
      </Text>

      <View className="flex-row gap-3 mb-8">
        {otp.map((digit, index) => (
          <TextInput
            key={index}
            ref={(ref) => { if (ref) inputRefs.current[index] =
ref; }}
            value={digit}
            onChangeText={(v) => handleOtpChange(v.slice(-1),
index)}
            keyboardType="number-pad"
            maxLength={1}
            className="flex-1 h-14 border-2 border-gray-300
rounded-xl text-center text-2xl font-bold text-gray-900
focus:border-green-500"
            autoFocus={index === 0}
          />
        ))}
      </View>

      {verifyOtp.isError && (
        <Text className="text-red-500 text-center mb-4">
          Invalid OTP. Please try again.
        </Text>
      )}

      <View className="items-center">
        {resendTimer > 0 ? (
          <Text className="text-gray-400">Resend OTP in
{resendTimer}s</Text>
        ) : (
          <Pressable onPress={() => {
            authService.requestGuardOtp(phone);
            setResendTimer(30);
          }}>
            <Text className="text-green-600 font-semibold">Resend
OTP</Text>
          </Pressable>
        )}
      </View>
    </View>
  );
}
```

---

4. Onboarding — Personal Info

```
// app/onboarding/personal-info.tsx
import { ScrollView, View, Text, TextInput, Pressable } from
```

```
'react-native';
import { useRouter } from 'expo-router';
import { useForm, Controller } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { z } from 'zod';
import { useMutation } from '@tanstack/react-query';
import { onboardingService } from '@/services/api/
onboardingService';

const SKILLS = [
  'Armed Guard',
  'Unarmed Guard',
  'CCTV Monitoring',
  'Event Security',
  'Personal Protection',
  'Retail Security',
  'Corporate Security',
  'Residential Security',
  'Fire Safety',
  'Access Control',
];

const schema = z.object({
  full_name: z.string().min(3, 'Full name must be at least 3
characters'),
  date_of_birth: z
    .string()
    .regex(/^\d{4}-\d{2}-\d{2}$/, 'Date must be in YYYY-MM-DD
format'),
  address: z.string().min(10, 'Please enter your full address'),
  city: z.string().min(2, 'City is required'),
  experience_years: z.coerce.number().min(0).max(50),
  skills: z.array(z.string()).min(1, 'Select at least one skill'),
});

type FormData = z.infer<typeof schema>;

export default function PersonalInfoScreen() {
  const router = useRouter();

  const {
    control,
    handleSubmit,
    watch,
    setValue,
    formState: { errors },
  } = useForm<FormData>({
    resolver: zodResolver(schema),
    defaultValues: { skills: [], experience_years: 0 },
  });

  const selectedSkills = watch('skills');

  const submitInfo = useMutation({
```

```
    mutationFn: onboardingService.submitPersonalInfo,
    onSuccess: () => router.push('/onboarding/documents'),
  });

  const toggleSkill = (skill: string) => {
    const current = selectedSkills || [];
    if (current.includes(skill)) {
      setValue('skills', current.filter((s) => s !== skill));
    } else {
      setValue('skills', [...current, skill]);
    }
  };

  return (
    <ScrollView className="flex-1 bg-white">
      <View className="px-6 pt-12 pb-8">
        {/* Header */}
        <View className="mb-2">
          <Text className="text-xs font-semibold text-green-600
uppercase tracking-widest mb-1">
            Step 1 of 3
          </Text>
          <Text className="text-3xl font-bold text-
gray-900">Personal Info</Text>
          <Text className="text-gray-500 mt-1">
            Tell us about yourself and your security expertise
          </Text>
        </View>

        {/* Progress Bar */}
        <View className="h-1.5 bg-gray-200 rounded-full mt-4
mb-8">
          <View className="h-full w-1/3 bg-green-500 rounded-
full" />
        </View>

        {/* Full Name */}
        <View className="mb-5">
          <Text className="text-sm font-medium text-gray-700
mb-1.5">
            Full Name *
          </Text>
          <Controller
            control={control}
            name="full_name"
            render={({ field: { onChange, value } }) => (
              <TextInput
                value={value}
                onChangeText={onChange}
                placeholder="As per your Aadhaar card"
                className="border border-gray-300 rounded-xl px-4
py-3.5 text-gray-900 text-base"
              />
            )}
```

```
          />
          {errors.full_name && (
            <Text className="text-red-500 text-xs
mt-1">{errors.full_name.message}</Text>
          )}
        </View>

        {/* Date of Birth */}
        <View className="mb-5">
          <Text className="text-sm font-medium text-gray-700
mb-1.5">
            Date of Birth *
          </Text>
          <Controller
            control={control}
            name="date_of_birth"
            render={({ field: { onChange, value } }) => (
              <TextInput
                value={value}
                onChangeText={onChange}
                placeholder="YYYY-MM-DD"
                keyboardType="numbers-and-punctuation"
                className="border border-gray-300 rounded-xl px-4
py-3.5 text-gray-900 text-base"
              />
            )}
          />
          {errors.date_of_birth && (
            <Text className="text-red-500 text-xs
mt-1">{errors.date_of_birth.message}</Text>
          )}
        </View>

        {/* Address */}
        <View className="mb-5">
          <Text className="text-sm font-medium text-gray-700
mb-1.5">
            Residential Address *
          </Text>
          <Controller
            control={control}
            name="address"
            render={({ field: { onChange, value } }) => (
              <TextInput
                value={value}
                onChangeText={onChange}
                placeholder="House/Flat No., Street, Area"
                multiline
                numberOfLines={3}
                className="border border-gray-300 rounded-xl px-4
py-3.5 text-gray-900 text-base"
                textAlignVertical="top"
              />
            )}
```

```
          />
          {errors.address && (
            <Text className="text-red-500 text-xs
mt-1">{errors.address.message}</Text>
          )}
        </View>

        {/* City */}
        <View className="mb-5">
          <Text className="text-sm font-medium text-gray-700
mb-1.5">City *</Text>
          <Controller
            control={control}
            name="city"
            render={({ field: { onChange, value } }) => (
              <TextInput
                value={value}
                onChangeText={onChange}
                placeholder="e.g. Mumbai, Delhi, Bangalore"
                className="border border-gray-300 rounded-xl px-4
py-3.5 text-gray-900 text-base"
              />
            )}
          />
          {errors.city && (
            <Text className="text-red-500 text-xs
mt-1">{errors.city.message}</Text>
          )}
        </View>

        {/* Experience Years */}
        <View className="mb-5">
          <Text className="text-sm font-medium text-gray-700
mb-1.5">
            Years of Experience
          </Text>
          <Controller
            control={control}
            name="experience_years"
            render={({ field: { onChange, value } }) => (
              <TextInput
                value={String(value)}
                onChangeText={onChange}
                placeholder="0"
                keyboardType="number-pad"
                className="border border-gray-300 rounded-xl px-4
py-3.5 text-gray-900 text-base"
              />
            )}
          />
        </View>

        {/* Skills */}
        <View className="mb-8">
```

```
          <Text className="text-sm font-medium text-gray-700
mb-1.5">
            Skills & Specializations *
          </Text>
          <Text className="text-xs text-gray-400 mb-3">Select all
that apply</Text>
          <View className="flex-row flex-wrap gap-2">
            {SKILLS.map((skill) => {
              const selected = selectedSkills?.includes(skill);
              return (
                <Pressable
                  key={skill}
                  onPress={() => toggleSkill(skill)}
                  className={`px-4 py-2 rounded-full border ${
                    selected
                      ? 'bg-green-600 border-green-600'
                      : 'bg-white border-gray-300'
                  }`}
                >
                  <Text
                    className={`text-sm font-medium ${
                      selected ? 'text-white' : 'text-gray-600'
                    }`}
                  >
                    {skill}
                  </Text>
                </Pressable>
              );
            })}
          </View>
          {errors.skills && (
            <Text className="text-red-500 text-xs
mt-2">{errors.skills.message}</Text>
          )}
        </View>

        {/* Submit */}
        <Pressable
          onPress={handleSubmit((data) =>
submitInfo.mutate(data))}
          disabled={submitInfo.isPending}
          className="bg-green-600 rounded-2xl py-4 items-center
active:opacity-80"
        >
          <Text className="text-white text-lg font-semibold">
            {submitInfo.isPending ? 'Saving...' : 'Continue to
Documents'}
          </Text>
        </Pressable>
      </View>
    </ScrollView>
  );
}
```

5. Onboarding — Documents

```
// app/onboarding/documents.tsx
import { useState } from 'react';
import { ScrollView, View, Text, Pressable, Image,
ActivityIndicator } from 'react-native';
import { useRouter } from 'expo-router';
import * as ImagePicker from 'expo-image-picker';
import { useDocumentUpload } from '@/hooks/useDocumentUpload';

interface DocumentItem {
  key: string;
  label: string;
  description: string;
  required: boolean;
}

const DOCUMENTS: DocumentItem[] = [
  {
    key: 'aadhaar_front',
    label: 'Aadhaar Card (Front)',
    description: 'Clear photo of the front side',
    required: true,
  },
  {
    key: 'aadhaar_back',
    label: 'Aadhaar Card (Back)',
    description: 'Clear photo of the back side',
    required: true,
  },
  {
    key: 'pan_card',
    label: 'PAN Card',
    description: 'Clear photo of your PAN card',
    required: true,
  },
  {
    key: 'police_verification',
    label: 'Police Verification Certificate',
    description: 'Character verification certificate',
    required: true,
  },
  {
    key: 'profile_photo',
    label: 'Profile Photo',
    description: 'Recent passport-size photo',
    required: true,
  },
];

export default function DocumentsScreen() {
  const router = useRouter();
  const [uploadedDocs, setUploadedDocs] = useState<Record<string,
```

```
string>>({});
  const [uploadingKey, setUploadingKey] = useState<string |
null>(null);
  const { uploadDocument } = useDocumentUpload();

  const handlePickAndUpload = async (docKey: string) => {
    const permission = await
ImagePicker.requestMediaLibraryPermissionsAsync();
    if (!permission.granted) return;

    const result = await ImagePicker.launchImageLibraryAsync({
      mediaTypes: ImagePicker.MediaTypeOptions.Images,
      quality: 0.85,
      allowsEditing: true,
      aspect: docKey === 'profile_photo' ? [1, 1] : [4, 3],
    });

    if (result.canceled || !result.assets[0]) return;

    const asset = result.assets[0];
    setUploadingKey(docKey);

    try {
      const s3Key = await uploadDocument({
        documentType: docKey,
        uri: asset.uri,
        mimeType: asset.mimeType || 'image/jpeg',
        fileName: asset.fileName || `${docKey}.jpg`,
      });
      setUploadedDocs((prev) => ({ ...prev, [docKey]: s3Key }));
    } catch (error) {
      console.error('Upload failed:', error);
    } finally {
      setUploadingKey(null);
    }
  };

  const allRequiredUploaded = DOCUMENTS.filter((d) =>
d.required).every(
    (d) => uploadedDocs[d.key]
  );

  return (
    <ScrollView className="flex-1 bg-white">
      <View className="px-6 pt-12 pb-8">
        {/* Header */}
        <Text className="text-xs font-semibold text-green-600
uppercase tracking-widest mb-1">
          Step 2 of 3
        </Text>
        <Text className="text-3xl font-bold text-gray-900
mb-1">Upload Documents</Text>
        <Text className="text-gray-500 mb-2">
          These will be verified by our team within 24–48 hours
```

```
        </Text>

        {/* Progress Bar */}
        <View className="h-1.5 bg-gray-200 rounded-full mt-4
mb-8">
          <View className="h-full w-2/3 bg-green-500 rounded-
full" />
        </View>

        {/* Document List */}
        <View className="gap-4 mb-8">
          {DOCUMENTS.map((doc) => {
            const isUploaded = !!uploadedDocs[doc.key];
            const isUploading = uploadingKey === doc.key;

            return (
              <View
                key={doc.key}
                className={`border rounded-2xl p-4 ${
                  isUploaded
                    ? 'border-green-300 bg-green-50'
                    : 'border-gray-200 bg-white'
                }`}
              >
                <View className="flex-row items-start justify-
between">
                  <View className="flex-1 mr-4">
                    <View className="flex-row items-center gap-2
mb-0.5">
                      <Text className="text-base font-semibold
text-gray-900">
                        {doc.label}
                      </Text>
                      {doc.required && (
                        <Text className="text-red-500 text-xs
font-medium">*</Text>
                      )}
                    </View>
                    <Text className="text-sm text-
gray-500">{doc.description}</Text>
                  </View>

                  {isUploading ? (
                    <ActivityIndicator color="#16a34a" />
                  ) : isUploaded ? (
                    <View className="w-8 h-8 bg-green-500 rounded-
full items-center justify-center">
                      <Text className="text-white text-lg">✓</
Text>
                    </View>
                  ) : (
                    <Pressable
                      onPress={() => handlePickAndUpload(doc.key)}
                      className="bg-green-600 rounded-xl px-4 py-2
```

```
active:opacity-80"
                    >
                      <Text className="text-white text-sm font-
medium">Upload</Text>
                    </Pressable>
                  )}
                </View>

                {isUploaded && (
                  <Pressable
                    onPress={() => handlePickAndUpload(doc.key)}
                    className="mt-3 pt-3 border-t border-
green-200"
                  >
                    <Text className="text-green-600 text-sm font-
medium">
                      Replace document
                    </Text>
                  </Pressable>
                )}
              </View>
            );
          })}
        </View>

        {/* Info Banner */}
        <View className="bg-amber-50 border border-amber-200
rounded-2xl p-4 mb-8">
          <Text className="text-amber-800 font-semibold mb-1">
            Document Guidelines
          </Text>
          <Text className="text-amber-700 text-sm leading-
relaxed">
            • Ensure all text is clearly legible{'\n'}
            • Photos must be taken in good lighting{'\n'}
            • Documents must be valid and not expired{'\n'}
            • File size limit: 5MB per document
          </Text>
        </View>

        {/* Submit */}
        <Pressable
          onPress={() => router.push('/onboarding/complete')}
          disabled={!allRequiredUploaded}
          className={`rounded-2xl py-4 items-center ${
            allRequiredUploaded ? 'bg-green-600
active:opacity-80' : 'bg-gray-200'
          }`}
        >
          <Text
            className={`text-lg font-semibold ${
              allRequiredUploaded ? 'text-white' : 'text-gray-400'
            }`}
          >
```

```
            Submit for Review
          </Text>
        </Pressable>
      </View>
    </ScrollView>
  );
}
```

---

6. Onboarding — Complete

```
// app/onboarding/complete.tsx
import { useEffect, useState } from 'react';
import { View, Text, Pressable } from 'react-native';
import { useRouter } from 'expo-router';
import { useQuery } from '@tanstack/react-query';
import { onboardingService } from '@/services/api/
onboardingService';

const TIMELINE_STEPS = [
  { key: 'submitted', label: 'Documents Submitted', description:
'Your documents have been received' },
  { key: 'under_review', label: 'Under Review', description: 'Our
team is verifying your documents' },
  { key: 'approved', label: 'Approved', description: 'You can
start accepting bookings!' },
];

export default function OnboardingCompleteScreen() {
  const router = useRouter();

  const { data: statusData } = useQuery({
    queryKey: ['verification-status'],
    queryFn: onboardingService.getVerificationStatus,
    refetchInterval: 30000, // Poll every 30 seconds
  });

  const currentStep = statusData?.status === 'approved'
    ? 2
    : statusData?.status === 'under_review'
    ? 1
    : 0;

  useEffect(() => {
    if (statusData?.status === 'approved') {
      const timer = setTimeout(() => router.replace('/(tabs)/
dashboard'), 2000);
      return () => clearTimeout(timer);
    }
  }, [statusData?.status]);

  return (
```

```
    <View className="flex-1 bg-white px-6 pt-20">
      {/* Success Icon */}
      <View className="items-center mb-8">
        <View className="w-24 h-24 bg-green-100 rounded-full
items-center justify-center mb-4">
          <Text className="text-5xl">✓</Text>
        </View>
        <Text className="text-2xl font-bold text-gray-900 text-
center mb-2">
          Documents Submitted!
        </Text>
        <Text className="text-gray-500 text-center leading-
relaxed">
          Our team will review your documents within 24–48 hours.
We'll notify you once approved.
        </Text>
      </View>

      {/* Timeline */}
      <View className="mb-10">
        {TIMELINE_STEPS.map((step, index) => {
          const isCompleted = index <= currentStep;
          const isActive = index === currentStep;

          return (
            <View key={step.key} className="flex-row items-start
mb-4">
              {/* Step Indicator */}
              <View className="items-center mr-4">
                <View
                  className={`w-10 h-10 rounded-full items-center
justify-center ${
                    isCompleted ? 'bg-green-500' : 'bg-gray-200'
                  }`}
                >
                  {isCompleted ? (
                    <Text className="text-white font-bold">✓</
Text>
                  ) : (
                    <Text className="text-gray-400 font-
bold">{index + 1}</Text>
                  )}
                </View>
                {index < TIMELINE_STEPS.length - 1 && (
                  <View
                    className={`w-0.5 h-8 mt-1 ${
                      index < currentStep ? 'bg-green-500' : 'bg-
gray-200'
                    }`}
                  />
                )}
              </View>

              {/* Step Content */}
```

```
              <View className="flex-1 pt-2">
                <Text
                  className={`font-semibold ${
                    isActive ? 'text-green-700' : isCompleted ?
'text-gray-900' : 'text-gray-400'
                  }`}
                >
                  {step.label}
                  {isActive && (
                    <Text className="text-green-500 font-normal
text-xs"> (Current)</Text>
                  )}
                </Text>
                <Text className="text-sm text-gray-500
mt-0.5">{step.description}</Text>
              </View>
            </View>
          );
        })}
      </View>

      {/* Info */}
      <View className="bg-blue-50 border border-blue-200
rounded-2xl p-4">
        <Text className="text-blue-800 font-semibold mb-1">What
happens next?</Text>
        <Text className="text-blue-700 text-sm leading-relaxed">
          You'll receive a push notification as soon as your
account is approved. This usually takes 24–48 hours on working
days.
        </Text>
      </View>
    </View>
  );
}
```

---

7. Dashboard Screen

```
// app/(tabs)/dashboard.tsx
import { useEffect, useRef } from 'react';
import {
  View,
  Text,
  Pressable,
  Animated,
  ScrollView,
  Switch,
} from 'react-native';
import { useGuardStore } from '@/stores/guardStore';
import { useActiveBookingStore } from '@/stores/
activeBookingStore';
```

```
import { useGuardStatus } from '@/hooks/useGuardStatus';
import { useIncomingBookingRequest } from '@/hooks/
useIncomingBookingRequest';
import { BookingRequestCard } from '@/components/booking/
BookingRequestCard';
import { RadarAnimation } from '@/components/dashboard/
RadarAnimation';
import { useRouter } from 'expo-router';

export default function DashboardScreen() {
  const router = useRouter();
  const { guard, isOnline } = useGuardStore();
  const { incomingRequest } = useActiveBookingStore();
  const { toggleOnlineStatus, isToggling } = useGuardStatus();

  // WebSocket subscription for incoming booking requests
  useIncomingBookingRequest();

  const requestCardAnim = useRef(new Animated.Value(300)).current;

  useEffect(() => {
    if (incomingRequest) {
      Animated.spring(requestCardAnim, {
        toValue: 0,
        useNativeDriver: true,
        tension: 65,
        friction: 11,
      }).start();
    } else {
      Animated.timing(requestCardAnim, {
        toValue: 300,
        duration: 300,
        useNativeDriver: true,
      }).start();
    }
  }, [incomingRequest]);

  return (
    <View className="flex-1 bg-gray-50">
      <ScrollView contentContainerStyle={{ flexGrow: 1 }}>
        {/* Header */}
        <View className="bg-green-950 pt-14 pb-8 px-6">
          <View className="flex-row items-center justify-between
mb-6">
            <View>
              <Text className="text-green-400 text-sm">Good
morning,</Text>
              <Text className="text-white text-xl font-bold">
                {guard?.full_name?.split(' ')[0] ?? 'Guard'}
              </Text>
            </View>
            <View className="items-end">
              <Text className="text-green-400 text-xs mb-1">
                {guard?.tier?.toUpperCase()} TIER
```

```
              </Text>
              <View className="flex-row items-center gap-1">
                <Text className="text-yellow-400 text-sm">★</Text>
                <Text className="text-white font-semibold">
                  {guard?.rating?.toFixed(1) ?? '5.0'}
                </Text>
              </View>
            </View>
          </View>

          {/* Online/Offline Toggle */}
          <View
            className={`rounded-3xl p-6 items-center ${
              isOnline ? 'bg-green-800' : 'bg-gray-800'
            }`}
          >
            <View
              className={`w-20 h-20 rounded-full items-center
justify-center mb-4 ${
                isOnline ? 'bg-green-500' : 'bg-gray-500'
              }`}
            >
              <Text className="text-4xl">{isOnline ? '🟢' : '⚫'}</
Text>
            </View>

            <Text className="text-white text-2xl font-bold mb-1">
              {isOnline ? 'You are Online' : 'You are Offline'}
            </Text>
            <Text
              className={`text-sm mb-6 ${
                isOnline ? 'text-green-300' : 'text-gray-400'
              }`}
            >
              {isOnline
                ? 'Accepting booking requests'
                : 'Toggle to start receiving requests'}
            </Text>

            <Switch
              value={isOnline}
              onValueChange={toggleOnlineStatus}
              disabled={isToggling}
              trackColor={{ false: '#374151', true: '#16a34a' }}
              thumbColor={isOnline ? '#ffffff' : '#9ca3af'}
              ios_backgroundColor="#374151"
              style={{ transform: [{ scaleX: 1.5 }, { scaleY:
1.5 }] }}
            />
          </View>
        </View>

        {/* Status Content */}
        <View className="flex-1 px-6 py-6">
```

```
          {isOnline ? (
            <View className="items-center py-8">
              <RadarAnimation />
              <Text className="text-gray-700 font-semibold text-lg
mt-4">
                Waiting for requests...
              </Text>
              <Text className="text-gray-400 text-sm text-center
mt-1">
                You'll be notified when a booking request comes in
nearby
              </Text>
            </View>
          ) : (
            <View className="items-center py-12">
              <Text className="text-6xl mb-4">😴</Text>
              <Text className="text-gray-600 font-semibold text-
lg">
                You're currently offline
              </Text>
              <Text className="text-gray-400 text-sm text-center
mt-2">
                Toggle the switch above to start receiving booking
requests
              </Text>
            </View>
          )}

          {/* Today's Stats */}
          <View className="flex-row gap-4 mt-4">
            <View className="flex-1 bg-white rounded-2xl p-4
shadow-sm">
              <Text className="text-gray-400 text-xs mb-1">Today's
Earnings</Text>
              <Text className="text-gray-900 text-xl font-bold">
                ₹{guard ? '0' : '--'}
              </Text>
            </View>
            <View className="flex-1 bg-white rounded-2xl p-4
shadow-sm">
              <Text className="text-gray-400 text-xs
mb-1">Bookings Today</Text>
              <Text className="text-gray-900 text-xl font-
bold">0</Text>
            </View>
          </View>
        </View>
      </ScrollView>

      {/* Incoming Request Bottom Sheet */}
      {incomingRequest && (
        <Animated.View
          style={{ transform: [{ translateY: requestCardAnim }] }}
          className="absolute bottom-0 left-0 right-0"
```

```
        >
          <BookingRequestCard
            request={incomingRequest}
            onAccept={() => router.push('/booking/active')}
            onDecline={() => {}}
          />
        </Animated.View>
      )}
    </View>
  );
}
```

---

8. Incoming Booking Request

```
// app/booking/request.tsx
import { useEffect, useRef, useState } from 'react';
import { View, Text, Pressable, Image, Animated } from 'react-
native';
import { useRouter } from 'expo-router';
import MapView, { Marker } from 'react-native-maps';
import { useActiveBookingStore } from '@/stores/
activeBookingStore';
import { useAcceptBooking } from '@/hooks/useAcceptBooking';

const COUNTDOWN_SECONDS = 30;

export default function BookingRequestScreen() {
  const router = useRouter();
  const { incomingRequest, setIncomingRequest } =
useActiveBookingStore();
  const [secondsLeft, setSecondsLeft] =
useState(COUNTDOWN_SECONDS);
  const progressAnim = useRef(new Animated.Value(1)).current;
  const { acceptBooking, declineBooking, isAccepting } =
useAcceptBooking(
    incomingRequest?.id ?? ''
  );

  useEffect(() => {
    // Countdown animation
    Animated.timing(progressAnim, {
      toValue: 0,
      duration: COUNTDOWN_SECONDS * 1000,
      useNativeDriver: false,
    }).start();

    const interval = setInterval(() => {
      setSecondsLeft((prev) => {
        if (prev <= 1) {
          clearInterval(interval);
          handleDecline();
```

```
          return 0;
        }
        return prev - 1;
      });
    }, 1000);

    return () => clearInterval(interval);
  }, []);

  const handleAccept = async () => {
    try {
      await acceptBooking();
      router.replace('/booking/active');
    } catch (error: any) {
      if (error?.response?.status === 409) {
        // Booking already taken by another guard
        setIncomingRequest(null);
        router.back();
      }
    }
  };

  const handleDecline = async () => {
    await declineBooking();
    setIncomingRequest(null);
    router.back();
  };

  if (!incomingRequest) return null;

  return (
    <View className="flex-1 bg-white">
      {/* Countdown Bar */}
      <Animated.View
        style={{
          height: 4,
          backgroundColor: '#16a34a',
          width: progressAnim.interpolate({
            inputRange: [0, 1],
            outputRange: ['0%', '100%'],
          }),
        }}
      />

      {/* Header */}
      <View className="bg-green-950 px-6 pt-12 pb-6">
        <View className="flex-row items-center justify-between
mb-4">
          <Text className="text-white text-xl font-bold">New
Booking Request</Text>
          <View className="w-12 h-12 bg-red-500 rounded-full
items-center justify-center">
            <Text className="text-white text-lg font-
bold">{secondsLeft}</Text>
```

```
          </View>
        </View>
        <Text className="text-green-300 text-sm">
          Auto-declining in {secondsLeft} seconds
        </Text>
      </View>

      {/* User Info */}
      <View className="flex-row items-center px-6 py-5 border-b
border-gray-100">
        <Image
          source={
            incomingRequest.user_photo
              ? { uri: incomingRequest.user_photo }
              : require('@/assets/images/default-avatar.png')
          }
          className="w-16 h-16 rounded-full bg-gray-200 mr-4"
        />
        <View className="flex-1">
          <Text className="text-lg font-bold text-gray-900">
            {incomingRequest.user_name}
          </Text>
          <View className="flex-row items-center gap-1 mt-0.5">
            <Text className="text-yellow-500 text-sm">★</Text>
            <Text className="text-gray-500 text-sm">
              {incomingRequest.user_rating?.toFixed(1)} rating
            </Text>
          </View>
          <Text className="text-gray-500 text-sm mt-0.5">
            {incomingRequest.booking_type}
          </Text>
        </View>
        <View className="items-end">
          <Text className="text-green-600 text-xl font-bold">
            ₹{incomingRequest.estimated_earnings}
          </Text>
          <Text className="text-gray-400 text-xs">estimated</Text>
        </View>
      </View>

      {/* Mini Map */}
      <MapView
        className="h-52"
        initialRegion={{
          latitude: incomingRequest.pickup_location.lat,
          longitude: incomingRequest.pickup_location.lon,
          latitudeDelta: 0.02,
          longitudeDelta: 0.02,
        }}
        scrollEnabled={false}
        zoomEnabled={false}
      >
        <Marker
          coordinate={{
```

```
            latitude: incomingRequest.pickup_location.lat,
            longitude: incomingRequest.pickup_location.lon,
          }}
          title="Pickup Location"
        />
      </MapView>

      {/* Location Details */}
      <View className="px-6 py-4 border-b border-gray-100">
        <Text className="text-xs font-semibold text-gray-400
uppercase tracking-wider mb-1">
          Pickup Location
        </Text>
        <Text className="text-gray-900 font-medium">
          {incomingRequest.pickup_address}
        </Text>
        <Text className="text-gray-500 text-sm mt-1">
          {incomingRequest.distance_km?.toFixed(1)} km from your
location
        </Text>
      </View>

      {/* Duration */}
      <View className="flex-row px-6 py-4 gap-6">
        <View>
          <Text className="text-xs text-gray-400 uppercase
tracking-wider mb-1">Duration</Text>
          <Text className="text-gray-900 font-
semibold">{incomingRequest.duration_hours}h</Text>
        </View>
        <View>
          <Text className="text-xs text-gray-400 uppercase
tracking-wider mb-1">Service Type</Text>
          <Text className="text-gray-900 font-
semibold">{incomingRequest.booking_type}</Text>
        </View>
      </View>

      {/* Action Buttons */}
      <View className="flex-row px-6 pb-8 gap-4 mt-auto">
        <Pressable
          onPress={handleDecline}
          className="flex-1 bg-red-100 rounded-2xl py-4 items-
center active:opacity-80"
        >
          <Text className="text-red-600 text-lg font-
semibold">Decline</Text>
        </Pressable>
        <Pressable
          onPress={handleAccept}
          disabled={isAccepting}
          className="flex-2 flex-1 bg-green-600 rounded-2xl py-4
items-center active:opacity-80"
          style={{ flex: 2 }}
```

```
        >
          <Text className="text-white text-lg font-semibold">
            {isAccepting ? 'Accepting...' : 'Accept ✓'}
          </Text>
        </Pressable>
      </View>
    </View>
  );
}
```

---

9. Active Booking Screen

```
// app/booking/active.tsx
import { useState, useEffect } from 'react';
import { View, Text, Pressable, Alert, Vibration } from 'react-
native';
import MapView, { Marker, Polyline } from 'react-native-maps';
import { useActiveBookingStore } from '@/stores/
activeBookingStore';
import { useBookingTimer } from '@/hooks/useBookingTimer';
import { useNavigationToUser } from '@/hooks/useNavigationToUser';
import { bookingService } from '@/services/api/bookingService';
import { useMutation } from '@tanstack/react-query';
import { useRouter } from 'expo-router';

type BookingStep = 'en_route' | 'arrived' | 'started' |
'completed';

const STEPS: { key: BookingStep; label: string; action: string }[]
= [
  { key: 'en_route', label: 'En Route', action: "I've Arrived" },
  { key: 'arrived', label: 'Arrived', action: 'Start Service' },
  { key: 'started', label: 'In Progress', action: 'Complete
Service' },
  { key: 'completed', label: 'Completed', action: '' },
];

export default function ActiveBookingScreen() {
  const router = useRouter();
  const { activeBooking, updateBookingStatus } =
useActiveBookingStore();
  const [currentStep, setCurrentStep] =
useState<BookingStep>('en_route');
  const elapsed = useBookingTimer(
    currentStep === 'started' ? activeBooking?.started_at :
undefined
  );
  const { navigateToUser } = useNavigationToUser(activeBooking);

  const markArrived = useMutation({
    mutationFn: () =>
```

```
bookingService.markArrived(activeBooking!.id),
    onSuccess: () => setCurrentStep('arrived'),
  });

  const startBooking = useMutation({
    mutationFn: () =>
bookingService.startBooking(activeBooking!.id),
    onSuccess: () => {
      setCurrentStep('started');
      updateBookingStatus('started');
    },
  });

  const completeBooking = useMutation({
    mutationFn: () =>
bookingService.completeBooking(activeBooking!.id),
    onSuccess: () => {
      setCurrentStep('completed');
      updateBookingStatus('completed');
      setTimeout(() => router.replace(`/booking/$
{activeBooking!.id}`), 2000);
    },
  });

  const handleSOSPress = () => {
    Vibration.vibrate([0, 500, 200, 500]);
    Alert.alert(
      'SOS Emergency',
      'Are you sure you want to trigger an SOS alert? Emergency
services and our team will be notified.',
      [
        { text: 'Cancel', style: 'cancel' },
        {
          text: 'Send SOS',
          style: 'destructive',
          onPress: () => {
            // Trigger SOS API call
          },
        },
      ]
    );
  };

  const handlePrimaryAction = () => {
    if (currentStep === 'en_route') markArrived.mutate();
    else if (currentStep === 'arrived') startBooking.mutate();
    else if (currentStep === 'started') {
      Alert.alert(
        'Complete Service',
        'Confirm that the service has been completed?',
        [
          { text: 'Cancel', style: 'cancel' },
          { text: 'Complete', onPress: () =>
completeBooking.mutate() },
```

```
        ]
      );
    }
  };

  const stepIndex = STEPS.findIndex((s) => s.key === currentStep);

  if (!activeBooking) return null;

  return (
    <View className="flex-1">
      {/* Full-Screen Map */}
      <MapView
        className="flex-1"
        showsUserLocation
        followsUserLocation={currentStep === 'en_route'}
        initialRegion={{
          latitude: activeBooking.pickup_location.lat,
          longitude: activeBooking.pickup_location.lon,
          latitudeDelta: 0.05,
          longitudeDelta: 0.05,
        }}
      >
        <Marker
          coordinate={{
            latitude: activeBooking.pickup_location.lat,
            longitude: activeBooking.pickup_location.lon,
          }}
          title={activeBooking.user_name}
          pinColor="#16a34a"
        />
      </MapView>

      {/* Status Steps Bar */}
      <View className="absolute top-14 left-4 right-4 bg-white
rounded-2xl p-4 shadow-lg">
        <View className="flex-row items-center justify-between">
          {STEPS.slice(0, 3).map((step, index) => (
            <View key={step.key} className="flex-row items-
center">
              <View
                className={`w-8 h-8 rounded-full items-center
justify-center ${
                  index <= stepIndex ? 'bg-green-500' : 'bg-
gray-200'
                }`}
              >
                {index < stepIndex ? (
                  <Text className="text-white text-xs font-
bold">✓</Text>
                ) : (
                  <Text
                    className={`text-xs font-bold ${
                      index === stepIndex ? 'text-white' : 'text-
```

```
gray-400'
                    }`}
                  >
                    {index + 1}
                  </Text>
                )}
              </View>
              {index < 2 && (
                <View
                  className={`h-0.5 w-12 mx-1 ${
                    index < stepIndex ? 'bg-green-500' : 'bg-
gray-200'
                  }`}
                />
              )}
            </View>
          ))}
        </View>

        <View className="flex-row items-center justify-between
mt-3">
          <View>
            <Text className="text-gray-900 font-
bold">{activeBooking.user_name}</Text>
            <Text className="text-gray-500 text-
sm">{activeBooking.booking_type}</Text>
          </View>
          {currentStep === 'started' && (
            <View className="items-end">
              <Text className="text-green-600 font-mono font-bold
text-lg">{elapsed}</Text>
              <Text className="text-gray-400 text-xs">elapsed</
Text>
            </View>
          )}
        </View>
      </View>

      {/* Bottom Action Panel */}
      <View className="absolute bottom-0 left-0 right-0 bg-white
rounded-t-3xl px-6 pt-6 pb-10 shadow-xl">
        {/* Navigate Button (shown during en_route) */}
        {currentStep === 'en_route' && (
          <Pressable
            onPress={navigateToUser}
            className="bg-blue-50 border border-blue-200
rounded-2xl py-3 items-center mb-4 flex-row justify-center gap-2"
          >
            <Text className="text-blue-600 font-semibold">Navigate
with Maps</Text>
          </Pressable>
        )}

        {/* Primary Action */}
```

```
        {currentStep !== 'completed' && (
          <Pressable
            onPress={handlePrimaryAction}
            className="bg-green-600 rounded-2xl py-4 items-center
mb-4 active:opacity-80"
          >
            <Text className="text-white text-lg font-semibold">
              {STEPS[stepIndex]?.action}
            </Text>
          </Pressable>
        )}

        {currentStep === 'completed' && (
          <View className="bg-green-50 rounded-2xl py-4 items-
center mb-4">
            <Text className="text-green-700 text-lg font-bold">
              Service Completed ✓
            </Text>
          </View>
        )}

        {/* SOS Button */}
        <Pressable
          onPress={handleSOSPress}
          className="bg-red-50 border border-red-200 rounded-2xl
py-3.5 items-center"
        >
          <Text className="text-red-600 font-bold text-base">
SOS Emergency</Text>
        </Pressable>
      </View>
    </View>
  );
}
```

---

10. Booking Detail

```
// app/booking/[id].tsx
import { ScrollView, View, Text, Image } from 'react-native';
import { useLocalSearchParams } from 'expo-router';
import { useQuery } from '@tanstack/react-query';
import { bookingService } from '@/services/api/bookingService';

export default function BookingDetailScreen() {
  const { id } = useLocalSearchParams<{ id: string }>();

  const { data: booking, isLoading } = useQuery({
    queryKey: ['booking', id],
    queryFn: () => bookingService.getBookingDetail(id),
  });
```

```
  if (isLoading || !booking) {
    return (
      <View className="flex-1 items-center justify-center">
        <Text className="text-gray-400">Loading...</Text>
      </View>
    );
  }

  const guardEarnings = booking.amount * 0.8; // 80% guard share

  return (
    <ScrollView className="flex-1 bg-gray-50">
      {/* Header */}
      <View className="bg-green-950 pt-14 pb-8 px-6">
        <Text className="text-green-400 text-sm mb-1">Booking
#{booking.id.slice(-6)}</Text>
        <Text className="text-white text-2xl font-bold mb-1">
          ₹{guardEarnings.toFixed(0)}
        </Text>
        <Text className="text-green-300 text-sm">Your earnings
from this booking</Text>
      </View>

      {/* User Info */}
      <View className="bg-white mx-4 mt-4 rounded-2xl p-4 shadow-
sm">
        <Text className="text-xs font-semibold text-gray-400
uppercase tracking-wider mb-3">
          Customer
        </Text>
        <View className="flex-row items-center">
          <Image
            source={{ uri: booking.user_photo }}
            className="w-12 h-12 rounded-full bg-gray-200 mr-3"
          />
          <View>
            <Text className="font-semibold text-
gray-900">{booking.user_name}</Text>
            <Text className="text-gray-500 text-
sm">{booking.user_rating?.toFixed(1)} ★</Text>
          </View>
        </View>
      </View>

      {/* Earnings Breakdown */}
      <View className="bg-white mx-4 mt-4 rounded-2xl p-4 shadow-
sm">
        <Text className="text-xs font-semibold text-gray-400
uppercase tracking-wider mb-4">
          Earnings Breakdown
        </Text>
        <View className="gap-3">
          <View className="flex-row justify-between">
            <Text className="text-gray-600">Service charges</Text>
```

```
            <Text className="text-gray-900 font-medium">₹
{booking.amount}</Text>
          </View>
          <View className="flex-row justify-between">
            <Text className="text-gray-600">Platform fee (20%)</
Text>
            <Text className="text-red-500 font-medium">- ₹
{(booking.amount * 0.2).toFixed(0)}</Text>
          </View>
          <View className="h-px bg-gray-100" />
          <View className="flex-row justify-between">
            <Text className="text-gray-900 font-bold">Your
earnings</Text>
            <Text className="text-green-600 font-bold text-lg">₹
{guardEarnings.toFixed(0)}</Text>
          </View>
        </View>
      </View>

      {/* Booking Timeline */}
      <View className="bg-white mx-4 mt-4 mb-8 rounded-2xl p-4
shadow-sm">
        <Text className="text-xs font-semibold text-gray-400
uppercase tracking-wider mb-4">
          Timeline
        </Text>
        {[
          { label: 'Request Accepted', time:
booking.accepted_at },
          { label: 'Arrived at Location', time:
booking.arrived_at },
          { label: 'Service Started', time: booking.started_at },
          { label: 'Service Completed', time:
booking.completed_at },
        ].map(({ label, time }) => (
          <View key={label} className="flex-row items-start mb-3">
            <View className="w-2 h-2 bg-green-500 rounded-full
mt-1.5 mr-3" />
            <View className="flex-1">
              <Text className="text-gray-900 font-medium">{label}
</Text>
              <Text className="text-gray-400 text-xs">
                {time ? new Date(time).toLocaleTimeString('en-IN',
{
                  hour: '2-digit',
                  minute: '2-digit',
                }) : '--'}
              </Text>
            </View>
          </View>
        ))}
      </View>
    </ScrollView>
```

```
  );
}
```

---

11. Earnings Tab

```
// app/(tabs)/earnings.tsx
import { ScrollView, View, Text, Pressable } from 'react-native';
import { useQuery, useMutation } from '@tanstack/react-query';
import { earningsService } from '@/services/api/earningsService';
import { VictoryBar, VictoryChart, VictoryAxis } from 'victory-
native';

const MIN_PAYOUT = 500;

export default function EarningsScreen() {
  const { data: summary } = useQuery({
    queryKey: ['earnings-summary'],
    queryFn: earningsService.getSummary,
  });

  const { data: payouts } = useQuery({
    queryKey: ['payout-history'],
    queryFn: earningsService.getPayoutHistory,
  });

  const requestPayout = useMutation({
    mutationFn: earningsService.requestPayout,
  });

  const canRequestPayout = (summary?.available_balance ?? 0) >=
MIN_PAYOUT;

  return (
    <ScrollView className="flex-1 bg-gray-50">
      {/* Header */}
      <View className="bg-green-950 pt-14 pb-8 px-6">
        <Text className="text-green-400 text-sm mb-1">Total
Earnings</Text>
        <Text className="text-white text-4xl font-bold">
          ₹{summary?.total?.toLocaleString('en-IN') ?? '0'}
        </Text>

        {/* Summary Cards */}
        <View className="flex-row gap-3 mt-6">
          <View className="flex-1 bg-green-900 rounded-2xl p-4">
            <Text className="text-green-400 text-xs mb-1">Today</
Text>
            <Text className="text-white text-xl font-bold">
              ₹{summary?.today ?? 0}
            </Text>
          </View>
```

```
          <View className="flex-1 bg-green-900 rounded-2xl p-4">
            <Text className="text-green-400 text-xs mb-1">This
Week</Text>
            <Text className="text-white text-xl font-bold">
              ₹{summary?.this_week?.toLocaleString('en-IN') ?? 0}
            </Text>
          </View>
        </View>
      </View>

      {/* Available Balance & Payout */}
      <View className="bg-white mx-4 mt-4 rounded-2xl p-5 shadow-
sm">
        <View className="flex-row items-center justify-between
mb-4">
          <View>
            <Text className="text-gray-500 text-sm">Available for
Payout</Text>
            <Text className="text-3xl font-bold text-gray-900
mt-1">
              ₹{summary?.available_balance?.toLocaleString('en-
IN') ?? 0}
            </Text>
          </View>
          <Pressable
            onPress={() => requestPayout.mutate()}
            disabled={!canRequestPayout ||
requestPayout.isPending}
            className={`rounded-2xl px-5 py-3 ${
              canRequestPayout ? 'bg-green-600' : 'bg-gray-200'
            }`}
          >
            <Text
              className={`font-semibold ${
                canRequestPayout ? 'text-white' : 'text-gray-400'
              }`}
            >
              {requestPayout.isPending ? 'Processing...' :
'Withdraw'}
            </Text>
          </Pressable>
        </View>
        {!canRequestPayout && (
          <Text className="text-gray-400 text-xs">
            Minimum payout: ₹{MIN_PAYOUT}. You need ₹
            {MIN_PAYOUT - (summary?.available_balance ?? 0)} more.
          </Text>
        )}
      </View>

      {/* Weekly Chart */}
      <View className="bg-white mx-4 mt-4 rounded-2xl p-5 shadow-
sm">
        <Text className="font-bold text-gray-900 mb-4">This Week</
```

```
Text>
        <VictoryChart height={200} padding={{ top: 10, bottom: 40,
left: 50, right: 20 }}>
          <VictoryAxis
            tickValues={['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat',
'Sun']}
            style={{ tickLabels: { fontSize: 10, fill:
'#9ca3af' } }}
          />
          <VictoryAxis
            dependentAxis
            style={{ tickLabels: { fontSize: 10, fill:
'#9ca3af' } }}
            tickFormat={(t) => `₹${t}`}
          />
          <VictoryBar
            data={summary?.weekly_chart ?? []}
            x="day"
            y="amount"
            style={{ data: { fill: '#16a34a', borderRadius: 4 } }}
            cornerRadius={{ top: 4 }}
          />
        </VictoryChart>
      </View>

      {/* Payout History */}
      <View className="mx-4 mt-4 mb-8">
        <Text className="font-bold text-gray-900 mb-3 px-1">Payout
History</Text>
        {payouts?.map((payout) => (
          <View
            key={payout.id}
            className="bg-white rounded-2xl p-4 mb-2 shadow-sm
flex-row items-center justify-between"
          >
            <View>
              <Text className="font-semibold text-gray-900">₹
{payout.amount}</Text>
              <Text className="text-gray-400 text-xs mt-0.5">
                {new
Date(payout.created_at).toLocaleDateString('en-IN')}
              </Text>
            </View>
            <View
              className={`px-3 py-1 rounded-full ${
                payout.status === 'completed'
                  ? 'bg-green-100'
                  : payout.status === 'pending'
                  ? 'bg-amber-100'
                  : 'bg-red-100'
              }`}
            >
              <Text
                className={`text-xs font-medium capitalize ${
```

```
                  payout.status === 'completed'
                    ? 'text-green-700'
                    : payout.status === 'pending'
                    ? 'text-amber-700'
                    : 'text-red-700'
                }`}
              >
                {payout.status}
              </Text>
            </View>
          </View>
        ))}
      </View>
    </ScrollView>
  );
}
```

---

12. History Tab

```
// app/(tabs)/history.tsx
import { useState } from 'react';
import { View, Text, FlatList, Pressable } from 'react-native';
import { useQuery } from '@tanstack/react-query';
import { bookingService } from '@/services/api/bookingService';
import { useRouter } from 'expo-router';

type FilterPeriod = 'today' | 'week' | 'month';

const FILTERS: { key: FilterPeriod; label: string }[] = [
  { key: 'today', label: 'Today' },
  { key: 'week', label: 'This Week' },
  { key: 'month', label: 'This Month' },
];

export default function HistoryScreen() {
  const router = useRouter();
  const [filter, setFilter] = useState<FilterPeriod>('week');

  const { data: bookings, isLoading } = useQuery({
    queryKey: ['booking-history', filter],
    queryFn: () => bookingService.getHistory(filter),
  });

  return (
    <View className="flex-1 bg-gray-50">
      {/* Header */}
      <View className="bg-white pt-14 pb-4 px-6 shadow-sm">
        <Text className="text-2xl font-bold text-gray-900
mb-4">History</Text>
        {/* Filter Pills */}
        <View className="flex-row gap-2">
```

```
          {FILTERS.map(({ key, label }) => (
            <Pressable
              key={key}
              onPress={() => setFilter(key)}
              className={`px-4 py-2 rounded-full ${
                filter === key ? 'bg-green-600' : 'bg-gray-100'
              }`}
            >
              <Text
                className={`text-sm font-medium ${
                  filter === key ? 'text-white' : 'text-gray-600'
                }`}
              >
                {label}
              </Text>
            </Pressable>
          ))}
        </View>
      </View>

      <FlatList
        data={bookings}
        keyExtractor={(item) => item.id}
        contentContainerStyle={{ padding: 16, gap: 8 }}
        renderItem={({ item }) => (
          <Pressable
            onPress={() => router.push(`/booking/${item.id}`)}
            className="bg-white rounded-2xl p-4 shadow-sm
active:opacity-80"
          >
            <View className="flex-row items-start justify-between
mb-2">
              <View className="flex-1">
                <Text className="font-semibold text-
gray-900">{item.user_name}</Text>
                <Text className="text-gray-500 text-
sm">{item.booking_type}</Text>
              </View>
              <Text className="text-green-600 font-bold text-lg">
                ₹{(item.amount * 0.8).toFixed(0)}
              </Text>
            </View>
            <View className="flex-row items-center justify-
between">
              <Text className="text-gray-400 text-xs">
                {new
Date(item.completed_at).toLocaleDateString('en-IN', {
                  day: 'numeric',
                  month: 'short',
                  hour: '2-digit',
                  minute: '2-digit',
                })}
              </Text>
              <View className="flex-row items-center gap-1">
```

```
                <Text className="text-yellow-500 text-xs">★</Text>
                <Text className="text-gray-400 text-xs">
                  {item.user_rating ?? 'Not rated'}
                </Text>
              </View>
            </View>
          </Pressable>
        )}
        ListEmptyComponent={
          !isLoading ? (
            <View className="items-center py-16">
              <Text className="text-4xl mb-3">  </Text>
              <Text className="text-gray-500">No bookings in this
period</Text>
            </View>
          ) : null
        }
      />
    </View>
  );
}
```

---

13. Profile Tab

```
// app/(tabs)/profile.tsx
import { ScrollView, View, Text, Pressable, Image } from 'react-
native';
import { useRouter } from 'expo-router';
import { useGuardStore } from '@/stores/guardStore';

const STATUS_CONFIG = {
  approved: { label: 'Verified', color: 'bg-green-100', textColor:
'text-green-700', dot: 'bg-green-500' },
  under_review: { label: 'Under Review', color: 'bg-amber-100',
textColor: 'text-amber-700', dot: 'bg-amber-500' },
  pending: { label: 'Pending', color: 'bg-amber-100', textColor:
'text-amber-700', dot: 'bg-amber-500' },
  unverified: { label: 'Unverified', color: 'bg-gray-100',
textColor: 'text-gray-600', dot: 'bg-gray-400' },
  rejected: { label: 'Rejected', color: 'bg-red-100', textColor:
'text-red-700', dot: 'bg-red-500' },
};

export default function ProfileScreen() {
  const router = useRouter();
  const { guard, clearGuard } = useGuardStore();

  if (!guard) return null;

  const statusConfig = STATUS_CONFIG[guard.verification_status];
```

```
  return (
    <ScrollView className="flex-1 bg-gray-50">
      {/* Header */}
      <View className="bg-green-950 pt-14 pb-10 px-6 items-
center">
        <Pressable onPress={() => router.push('/documents/
upload')}>
          <Image
            source={
              guard.photo_url
                ? { uri: guard.photo_url }
                : require('@/assets/images/default-avatar.png')
            }
            className="w-24 h-24 rounded-full bg-green-800 mb-3"
          />
          <Text className="text-green-400 text-xs text-
center">Change Photo</Text>
        </Pressable>

        <Text className="text-white text-2xl font-bold
mt-3">{guard.full_name}</Text>
        <Text className="text-green-400 mb-3">{guard.city}</Text>

        {/* Verification Badge */}
        <View className={`flex-row items-center gap-2 px-4 py-1.5
rounded-full ${statusConfig.color}`}>
          <View className={`w-2 h-2 rounded-full $
{statusConfig.dot}`} />
          <Text className={`text-sm font-medium $
{statusConfig.textColor}`}>
            {statusConfig.label}
          </Text>
        </View>
      </View>

      {/* Stats */}
      <View className="flex-row mx-4 mt-4 gap-3">
        {[
          { label: 'Rating', value: `${guard.rating?.toFixed(1) ??
'5.0'} ★` },
          { label: 'Bookings', value:
String(guard.total_bookings) },
          { label: 'Tier', value: guard.tier?.toUpperCase() ??
'BASIC' },
        ].map(({ label, value }) => (
          <View key={label} className="flex-1 bg-white rounded-2xl
p-4 items-center shadow-sm">
            <Text className="text-gray-900 font-bold text-
lg">{value}</Text>
            <Text className="text-gray-400 text-xs mt-0.5">{label}
</Text>
          </View>
        ))}
      </View>
```

```
      {/* Menu Items */}
      <View className="mx-4 mt-4 bg-white rounded-2xl shadow-sm
overflow-hidden">
        {[
          { label: 'Edit Profile', route: '/onboarding/personal-
info', icon: ' ' },
          { label: 'My Documents', route: '/onboarding/documents',
icon: ' ' },
          { label: 'Availability Schedule', route: null, icon:
' ' },
          { label: 'Earnings History', route: null, icon: ' ' },
        ].map(({ label, route, icon }, index) => (
          <Pressable
            key={label}
            onPress={() => route && router.push(route as any)}
            className={`flex-row items-center px-5 py-4 active:bg-
gray-50 ${
              index > 0 ? 'border-t border-gray-100' : ''
            }`}
          >
            <Text className="text-xl mr-3">{icon}</Text>
            <Text className="flex-1 text-gray-900 font-
medium">{label}</Text>
            <Text className="text-gray-400">›</Text>
          </Pressable>
        ))}
      </View>

      {/* Skills */}
      <View className="mx-4 mt-4 bg-white rounded-2xl p-4 shadow-
sm">
        <Text className="font-semibold text-gray-900
mb-3">Skills</Text>
        <View className="flex-row flex-wrap gap-2">
          {guard.skills?.map((skill) => (
            <View key={skill} className="bg-green-50 border
border-green-200 px-3 py-1.5 rounded-full">
              <Text className="text-green-700 text-sm font-
medium">{skill}</Text>
            </View>
          ))}
        </View>
      </View>

      {/* Logout */}
      <Pressable
        onPress={() => {
          clearGuard();
          // router.replace('/(auth)/welcome')
        }}
        className="mx-4 mt-4 mb-8 bg-red-50 border border-red-100
rounded-2xl py-4 items-center"
      >
```

```
        <Text className="text-red-600 font-semibold">Sign Out</
Text>
      </Pressable>
    </ScrollView>
  );
}
```